MODULE 09 · DOMAIN 1 - DATA PREPARATION FOR ML AND AI

Embeddings and vector stores

Turning meaning into numbers, and finding the nearest ones fast.

MODULE 09 of 43 DOMAIN 1 - Data Prep WEIGHT 28%

admin@sthithapragna.com

An embedding turns text, an image or audio into a list of numbers where **distance means similarity**.

What embeddings let you do that keywords cannot

- **Semantic search.** A query about 'cancelling my subscription' finds a document titled 'ending your plan'.
- Keyword search finds neither, because the two share almost no words.
- **Clustering and deduplication** by meaning rather than by string match.
- **Recommendation** by similarity between items or users.
- **Retrieval for RAG**, which is why this appears in an ML engineering exam at all.

> ON THE EXAM
>
> Module 05's TF-IDF matches words. Embeddings match meaning. That is the whole upgrade.

Things that are true of every embedding

- **Fixed length.** A three-word query and a long paragraph produce the same-sized vector.
- **Dense.** Every dimension carries some signal, unlike one-hot's sea of zeros.
- **Not interpretable.** Dimension 417 does not mean anything you can name.
- **Model-specific.** Vectors from two different models are not comparable at all.
- **Derived.** If the source text changes, the vector is stale until you regenerate it.

> ON THE EXAM
>
> The last two cause most real problems. Vectors are derived state, and they belong to the model that made them.

Query and documents must be embedded by the **same model**, with the **same settings**.

Embedding models on Bedrock

MODEL	NOTES
Amazon Titan Text Embeddings V2	1024, 512 or 256 dimensions. 100+ languages. The default choice
Amazon Titan Multimodal Embeddings	Text and images into one shared space. Search images with text
Cohere Embed	Multilingual variants, strong retrieval performance
Third-party and open models	Available via SageMaker JumpStart if you need a specific one

ON THE EXAM

Multimodal models put images and text in one space, so a text query can retrieve a photograph. That is not possible with two separate models.

Multimodal embeddings

- Text and images are embedded into **one shared space** by a single model.
- So a text query can retrieve an image, and an image can retrieve similar images.
- Search a product catalogue by description, or find visually similar items from a photo.
- You cannot achieve this by embedding text and images with two separate models.
- Audio is usually transcribed first, then embedded as text, unless a model handles it directly.

> ON THE EXAM
>
> This is the payoff for module 02's 'ingest text, images and audio'. Different media, one searchable space.

Titan Text Embeddings V2

- **Output dimensions: 1024 by default, or 512, or 256.** You choose per request.
- **512 retains about 99 percent** of retrieval accuracy. **256 retains about 97 percent.**
- Input up to **8,192 tokens**, roughly fifty thousand characters.
- Over 100 languages, and optional **unit vector normalisation**.
- Latency-optimised invocation for retrieval; throughput-optimised batch jobs for indexing.

> ON THE EXAM
>
> Those accuracy figures are the whole dimension argument. Half the storage for one percent of accuracy is usually an easy trade.

Generating embeddings at scale

PATH	WHEN TO USE IT
`InvokeModel, one at a time`	Query embedding at request time. Latency-optimised
`Bedrock batch inference job`	Indexing a whole corpus. Throughput-optimised, reads JSONL from S3
`Bedrock Knowledge Bases`	It embeds for you during ingestion. No code at all
`SageMaker AI batch transform`	When the embedding model is hosted on SageMaker AI rather than Bedrock

ON THE EXAM

Embedding a corpus and embedding a query are different workloads. One is throughput-bound, the other latency-bound.

The economics of building an index

- Bedrock charges embedding models **per input token**. There is no output token charge.
- Indexing a corpus is a **large one-off cost**. Embedding queries is a small recurring one.
- Re-embedding after a model change repeats the whole one-off cost. Budget for it before you commit.
- Batch inference is cheaper per token than real-time invocation for bulk indexing.
- Then storage and query costs follow, which is the vector store's bill rather than Bedrock's.

> ON THE EXAM
>
> Two bills, not one: Bedrock for creating the vectors, and the vector store for keeping and searching them.

Smaller vectors are cheaper in four ways at once

- **Less storage.** 256 dimensions is a quarter of the bytes of 1024.
- **Smaller index**, which reduces memory in stores that hold the index in RAM.
- **Faster distance calculations**, so lower query latency at the same scale.
- **Lower query cost** where pricing is based on index size, as with S3 Vectors.
- You pay in accuracy, but the published figures make it a modest price.

> ON THE EXAM
>
> Dimension is not a modelling preference. On a large corpus it is one of the biggest cost levers you have.

Binary embeddings and quantisation

- **Binary embeddings** encode each dimension as a single bit rather than a float.
- That is a very large storage and compute saving, and distance becomes bitwise.
- Titan Text Embeddings V2 supports binary output, alone or alongside float.
- **FP16** halves storage against FP32 with very little accuracy loss.
- Quantisation always trades some precision. Measure retrieval quality, do not assume.

> ON THE EXAM
>
> Common pattern: search a binary index fast to get candidates, then rescore the top results with full-precision vectors.

Three distance metrics

METRIC	WHAT IT MEASURES
Cosine similarity	The angle between vectors. Ignores magnitude. The usual choice for text
Euclidean (L2)	Straight-line distance. Magnitude matters
Dot product	Angle and magnitude together. Fast, and sensitive to vector length

ON THE EXAM

On normalised vectors, cosine and dot product rank results identically. Normalisation is why the choice often stops mattering.

Comparing a query against ten million vectors **one by one** is correct, and far too slow.

How approximate search is actually done

INDEX	BEHAVIOUR
Flat / brute force	Compares everything. Exact, slow, fine for small collections
HNSW	A navigable graph. Fast and accurate, holds the graph in memory
IVF	Clusters vectors, searches only the nearest clusters. Lower memory
Product quantisation	Compresses vectors within an index. Often combined with IVF

ON THE EXAM

HNSW is the one to know. It is what OpenSearch Serverless uses through FAISS, and what most managed stores default to.

The recall and latency dial

- Every ANN index has parameters trading **recall against speed**.
- In HNSW, more graph connections and a wider search give better recall and more work.
- **Recall here means 'did we find the true nearest neighbours'**, not the classification metric.
- Low retrieval quality is often an index tuning problem, not an embedding problem.
- Managed services hide most of this. Knowing the dial exists is usually enough.

> ON THE EXAM
>
> Two different things are called recall in this exam. Module 17's recall is about classification. This one is about search.

The exam guide names three: **Amazon OpenSearch Service**, **Amazon RDS with pgvector**, and **Amazon S3**.

Amazon S3 Vectors

- A new bucket type, the **vector bucket**, holding **vector indexes**. Dedicated APIs.
- Purpose-built for cost. AWS claims **up to 90 percent lower** total cost than specialised vector databases.
- No infrastructure to provision, and **strongly consistent writes**.
- **Natively integrated with Bedrock Knowledge Bases.**
- General availability December 2025, and in 31 Regions since March 2026.

> ON THE EXAM
>
> This is the answer when a scenario is dominated by cost and scale rather than by latency.

S3 Vectors limits worth knowing

LIMIT	VALUE
Vectors per index	2 billion
Indexes per vector bucket	10,000, up to 20 trillion vectors per bucket
Query latency	Under 1 second infrequent; around 100 ms or less when frequent
Write throughput	1,000 vectors per second, streaming single-vector updates
Results per query	Up to 100
Metadata keys per vector	Up to 50, filterable by default

ON THE EXAM

The preview allowed 50 million vectors per index. Anything quoting that figure predates GA by a factor of forty.

What is fixed at index creation

- **Dimension** is fixed. You cannot change it later.
- **Distance metric** is fixed at creation too.
- **Non-filterable metadata keys** are declared at creation and cannot change.
- Changing any of these means creating a **new index** and reloading.
- Non-filterable keys are excluded from the index size that query pricing is based on.

> ON THE EXAM
>
> Reloading is not a disaster: your source documents are in S3, so the index is derived state you can rebuild.

S3 Vectors pricing has three dimensions

- **PUT**, charged on the logical gigabytes you upload, including metadata and keys.
- **Storage**, charged on total logical storage across your indexes.
- **Query**, a per-API charge **plus a per-terabyte charge based on index size**.
- The per-terabyte rate improves as an index grows beyond 100,000 vectors.
- Query cost scales with index size, so dimension choice shows up directly on the bill.

> ON THE EXAM
>
> This is why the dimension slide mattered. Halving dimensions halves the index size that every query is charged against.

Amazon OpenSearch Service

- Vector search through the **k-NN plugin**, alongside full-text and filtering.
- **Hybrid search** is its distinctive strength: combine semantic and keyword scoring.
- Serverless uses the **VECTORSEARCH collection type** with FAISS HNSW.
- **NextGen**, GA May 2026, adds **scale-to-zero** on a ten-minute idle timeout.
- Waking from zero costs **10 to 30 seconds** on the first query. Not for latency-critical paths.

> ON THE EXAM
>
> Hybrid search is often better than pure vector search. Exact identifiers and product codes are exactly what embeddings handle worst.

How hybrid search actually works

- Run a **vector query** and a **keyword query** over the same corpus, then combine the scores.
- The two score on different scales, so they must be **normalised** before combining.
- You weight the two sides. More keyword weight for identifiers, more vector weight for questions.
- It fixes the two failure modes at once: embeddings miss exact codes, keywords miss paraphrase.
- OpenSearch supports this natively through search pipelines with normalisation processors.

> ON THE EXAM
>
> If a scenario mentions both natural-language questions and exact product or part numbers, hybrid is the intended answer.

Amazon RDS and Aurora with pgvector

- **pgvector** is a PostgreSQL extension adding a vector type and index support.
- The case: **your data is already in PostgreSQL** and you want one system, not two.
- You can **join vectors to relational rows** in a single SQL query. Nothing else here does that.
- Full transactional consistency between the vector and the record it describes.
- It scales with your database. Very large corpora are not what it is best at.

> ON THE EXAM
>
> The joinability argument is genuine. Filtering by tenant, permissions and date in SQL alongside a similarity search is a real advantage.

Other AWS services that store vectors

SERVICE	THE CASE FOR IT
Amazon Aurora PostgreSQL	pgvector, with Aurora's scaling and read replicas
Amazon MemoryDB	In-memory vector search. Single-digit millisecond, highest cost
Amazon DocumentDB	Vector search for teams already on a document database
Amazon Neptune Analytics	Vectors alongside a graph, for relationship-aware retrieval
Amazon OpenSearch Serverless	The managed, scale-to-zero form of OpenSearch

ON THE EXAM

The guide names OpenSearch, RDS with pgvector and S3. Recognise the others, but answer with the named three unless a scenario demands otherwise.

Which vector store

THE SITUATION	THE STORE
Very large corpus, cost is the driver	Amazon S3 Vectors
Infrequent or bursty queries, sub-second is fine	Amazon S3 Vectors
Low latency at high query rates	Amazon OpenSearch Service
Semantic plus keyword in one query	OpenSearch, using hybrid search
Vectors must join to relational data	RDS or Aurora with pgvector
Team already runs PostgreSQL, modest scale	RDS or Aurora with pgvector
Billions of vectors, hot subset needs speed	S3 Vectors tiered with OpenSearch

ON THE EXAM

Latency and cost pull in opposite directions here. Read the scenario for which one it actually cares about.

The tiered pattern

- Keep the **bulk of the corpus in S3 Vectors**, where storage is cheap.
- Promote the **frequently queried subset into OpenSearch** for low latency.
- S3 Vectors integrates with OpenSearch Service specifically to support this.
- Same logic as S3 storage classes: cost follows access frequency.
- Two stores means two things to keep in sync. Do not reach for it without the scale to justify it.

> ON THE EXAM
>
> This is the answer when a scenario states both a huge corpus and a strict latency requirement, and refuses to compromise on either.

Metadata filtering, and why it matters

- Store attributes alongside each vector: source, date, department, tenant, permissions.
- **Filter before or during search**, so results respect who is asking.
- Without it, a semantic search happily returns documents the user may not read.
- S3 Vectors allows up to **50 metadata keys** per vector, filterable by default.
- Non-filterable keys are stored but excluded from filtering, and from query pricing.

> ON THE EXAM
>
> Access control in a RAG system usually lives here. Getting it wrong leaks documents through an interface nobody audited.

Multi-tenancy in a vector store

- **One index per tenant** is the cleanest isolation. S3 Vectors allows 10,000 per bucket.
- IAM can then restrict by index ARN, so isolation is enforced outside your code.
- **A dedicated KMS key per index** is supported, for regulated multi-tenant work.
- One shared index with a tenant metadata filter relies entirely on your filter being correct.
- In pgvector, row-level security does the same job through PostgreSQL.

> ON THE EXAM
>
> Isolation enforced by IAM survives a bug in your query code. Isolation enforced by a WHERE clause does not.

Embeddings are not anonymised data

- A vector looks like meaningless numbers. It is not a privacy control.
- **Embedding inversion** research shows source text can be partially reconstructed from vectors.
- Treat an embedding as **as sensitive as the text it came from**, and protect it the same way.
- Encrypt vector stores, apply the same access controls, and honour the same residency rules.
- Chunks often carry the source text in metadata anyway, in plain form.

> ON THE EXAM
>
> Module 08's rule again: hashing was not anonymisation, and neither is embedding. The obligations travel with the data.

Vectors are derived state

- A vector index is **rebuildable from your source documents**. Treat it that way.
- There is no obvious backup story for vector stores, and there does not need to be.
- **Changing embedding model means re-embedding everything.** Not a partial migration.
- Changed documents need re-embedding too, or retrieval quietly returns stale content.
- Keep the raw documents in S3, and make the rebuild a repeatable job.

> ON THE EXAM
>
> Module 06's lesson again: something has to keep this current, and nothing alarms when it stops.

Measuring retrieval quality

- **Recall at k:** of the documents that should have been retrieved, how many appeared in the top k?
- **Precision at k:** of the k returned, how many were actually relevant?
- **MRR** and **NDCG** reward putting the right answer near the top, not merely in the list.
- All of these need a **labelled set of queries and their correct documents**. Someone has to build it.
- Retrieval quality is separate from answer quality. Module 18 covers evaluating both.

> ON THE EXAM
>
> A RAG system fails at retrieval far more often than at generation. Measure the retrieval step on its own.

Symptoms, and what they usually mean

SYMPTOM	USUAL CAUSE
Results look random, no errors anywhere	Query and documents embedded by different models
First query after idle takes 20 seconds	OpenSearch Serverless waking from scale-to-zero
Retrieval misses documents you know exist	ANN recall too low, or a metadata filter excluding them
Users see documents they should not	No metadata filter on permissions during search
Answers are correct but outdated	Documents changed and were never re-embedded
Query bill rises as the corpus grows	S3 Vectors query charges scale with index size
Cannot change the index dimension	Fixed at creation. Create a new index and reload

ON THE EXAM

The first row is the important one. Nothing fails, nothing logs, and the system confidently returns nonsense.

The embedding model defines the space. **Everything in it must come from the same model.**

> A media archive is building semantic search over 700 million document chunks. Queries arrive in bursts during working hours and are rare overnight, at roughly 40,000 queries per day.
>
> Sub-second response is acceptable. The finance team has made cost the deciding factor, and the archive grows by several million chunks each month.

Which vector store fits best?

A Amazon OpenSearch Service provisioned cluster with the k-NN plugin

B Amazon S3 Vectors, using vector buckets and vector indexes

C Amazon RDS for PostgreSQL with the pgvector extension

D Amazon OpenSearch Serverless Classic with a VECTORSEARCH collection

◆ PAUSE AND DECIDE

Which vector store fits best?

A. Amazon OpenSearch Service provisioned cluster with the k-NN plugin

Would perform well and bills continuously for 700 million vectors, including overnight when queries are rare. Cost is the stated driver.

B. Amazon S3 Vectors, using vector buckets and vector indexes

Purpose-built for exactly this: billion-scale, cost-dominated, sub-second acceptable. 700 million fits in one index, and growth needs no re-provisioning.

C. Amazon RDS for PostgreSQL with the pgvector extension

pgvector scales with the database instance. 700 million vectors and millions added monthly is well outside where it is comfortable.

D. Amazon OpenSearch Serverless Classic with a VECTORSEARCH collection

Classic keeps the old OCU floor, so it bills a minimum even when idle. NextGen with scale-to-zero would be the better half of this idea.

THE SIGNAL

Three facts point one way: enormous scale, bursty low-volume queries, and cost as the stated deciding factor.

A support knowledge base has run well for eight months. To improve multilingual coverage, a team switches the embedding model used for incoming queries from Titan Text Embeddings V2 to a Cohere multilingual model.

After deployment, retrieved passages are unrelated to the questions asked. There are no errors, similarity scores look normal, and latency is unchanged.

What happened?

A The new model returns a different dimension, so the index rejected the queries

B The distance metric must be changed from cosine to dot product for Cohere

C The ANN index needs its recall parameters retuned after a model change

D Queries and stored documents are now embedded by different models, so the spaces do not match

◆ PAUSE AND DECIDE

What happened?

A The new model returns a different dimension, so the index rejected the queries

A dimension mismatch produces an error. The scenario states there are no errors, so queries are being accepted and answered.

B The distance metric must be changed from cosine to dot product for Cohere

Metric choice affects ranking at the margins. It cannot turn relevant results into unrelated ones.

C The ANN index needs its recall parameters retuned after a model change

Low recall misses some true neighbours. It does not return unrelated passages, and retuning cannot fix a space mismatch.

D Queries and stored documents are now embedded by different models, so the spaces do not match

Two models place meaning in unrelated coordinate systems. Nearest neighbours in the wrong space are effectively random, and nothing errors.

THE SIGNAL

The fix is not a setting. The entire corpus must be re-embedded with the new model, because the index is derived state.

> An e-commerce site needs product search returning results in under 100 milliseconds at the 99th percentile, at several thousand queries per second during sales.
>
> Searches must combine meaning with exact matching on product codes and SKUs, and must filter by category, availability and price range in the same request.

Which approach fits?

A Amazon OpenSearch Service with hybrid search and filters on a provisioned cluster

B Amazon S3 Vectors with metadata filtering on category, availability and price

C Aurora PostgreSQL with pgvector and SQL filters on the same query

D Amazon S3 Vectors tiered with OpenSearch Serverless NextGen

◆ PAUSE AND DECIDE

Which approach fits?

A Amazon OpenSearch Service with hybrid search and filters on a provisioned cluster

Hybrid search is the only option that scores meaning and exact keyword matches together, and provisioned capacity sustains p99 under 100 ms at high QPS.

B Amazon S3 Vectors with metadata filtering on category, availability and price

Filtering is fine, but S3 Vectors targets around 100 ms when warm, not p99 under 100 ms. It also has no keyword scoring for SKUs.

C Aurora PostgreSQL with pgvector and SQL filters on the same query

Joins and filters are its strength. Several thousand QPS with a p99 latency target is not, and it lacks full-text relevance scoring.

D Amazon S3 Vectors tiered with OpenSearch Serverless NextGen

Tiering suits huge corpora, and a product catalogue is not huge. Scale-to-zero also risks a 10-30 second first query during a sale.

THE SIGNAL

SKU matching is the quiet requirement. Embeddings are worst at exact identifiers, which is precisely what hybrid search exists to fix.

A SaaS company stores case records for 300 customer organisations in Aurora PostgreSQL. They want semantic search over case notes, roughly 4 million records in total.

Every search must be strictly scoped to the requesting organisation, and results must be filtered by case status and assigned team, both of which live in existing relational tables.

What is the most appropriate design?

A Export notes to S3 Vectors with one index per organisation, restricted by IAM

B Load notes into OpenSearch Serverless with an organisation metadata filter

C Add pgvector to the existing Aurora cluster and query vectors alongside the relational tables

D Use S3 Vectors for the notes and join results back to Aurora in application code

◆ PAUSE AND DECIDE

What is the most appropriate design?

A. Export notes to S3 Vectors with one index per organisation, restricted by IAM

Strong isolation, and it splits the data. Status and team live in Aurora, so every search still needs a second query and a join.

B. Load notes into OpenSearch Serverless with an organisation metadata filter

Workable, and it adds a second datastore to keep in sync for 4 million records, with filters duplicated outside the database that owns them.

C. Add pgvector to the existing Aurora cluster and query vectors alongside the relational tables

4 million records is comfortable for pgvector, and similarity, tenant scoping, status and team resolve in one SQL statement against one source of truth.

D. Use S3 Vectors for the notes and join results back to Aurora in application code

Moves the join into application code, where filtering after retrieval can return too few results and tenant scoping is enforced by your code, not the database.

THE SIGNAL

Four million records is modest, and the filters live in relational tables. Both facts point at keeping one system rather than two.

What to take from this module

- **One model, one space.** Query and documents must share an embedding model, or results are noise.
- **S3 Vectors for cost and scale. OpenSearch for latency and hybrid. pgvector for joins.**
- **Dimension is a cost lever.** 512 keeps ~99 percent of accuracy, 256 keeps ~97 percent.
- **Dimension and distance metric are fixed at index creation.** Changing them means a new index.
- **A vector index is derived state.** Keep the source documents and make rebuilds routine.

> ON THE EXAM
>
> Module 10 covers what you embed: chunking, metadata and preparing documents for retrieval.